

مشروع طلاب

إنشاء نموذج مصغر للتصنيف العمراني
والذكاء الاصطناعي OpenStreetMap باستخدام بيانات

Custom MLLM-Geo-AI

"A Lightweight Multi-Modal GeoAI Model for Urban Digital Twin Using OSM POI, Road Networks, and Satellite Imager

المرحلة 2: جمع البيانات (Multi-Modal Data)

3 تحميل شبكة الطرق باستخدام OSMnx

تثبيت المكتبات OSMnx

```
pip install osmnx
يفضل أيضاً:
pip install geopandas networkx matplotlib
```

استدعاء المكتبات

```
import osmnx as ox
import networkx as nx
import geopandas as gpd
import matplotlib.pyplot as plt
```

تحديد منطقة الدراسة

باستخدام اسم مكان: (A) الطريقة
place = "Cairo, Egypt"

الطريقة (B): Bounding Box (أفضل)

```
north = 30.10
south = 29.90
east = 31.30
west = 31.10
```

تحميل شبكة الطرق

من اسم المكان: الطريقة (A)

```
G = ox.graph_from_place(place, network_type="drive")
```

الطريقة (B): من Bounding Box

```
G = ox.graph_from_bbox(north, south, east, west, network_type="drive")
```

اختيار نوع الشبكة (مهم جداً)

النوع	الاستخدام
drive	سيارات
walk	مشاة
bike	دراجات
all	كل الطرق

مثال:

```
G = ox.graph_from_place(place, network_type="walk")
```

فهم مكونات الـ Graph الشبكة الناتجة:

Graph G

تتكون من:

Nodes (العقد)

- تقاطعات الطرق
- تحتوي:
 - latitude
 - longitude

Edges (الحواف)

- الطرق بين العقد
- تحتوي:
 - length
 - highway type
 - name

استكشاف البيانات

عدد العقد والطرق

```
print(len(G.nodes), len(G.edges))
```

عرض معلومات عقدة

```
node = list(G.nodes)[0]
print(G.nodes[node])
```

عرض معلومات طريق

```
edge = list(G.edges)[0]
print(G.edges[edge])
```

```
ox.plot_graph(G)
```

```
nodes, edges = ox.graph_to_gdfs(G)
```

- nodes → نقاط
- edges → خطوط

حفظ Shapefile

```
nodes.to_file("nodes.shp")
edges.to_file("edges.shp")
```

حفظ GraphML (مهم للـ GNN)

```
ox.save_graphml(G, "road_network.graphml")
```

```
G = ox.load_graphml("road_network.graphml")
```

Edge Index تحويل إلى

```
edge_index = list(G.edges)
```

adjacency matrix تحويل إلى

```
A = nx.adjacency_matrix(G)
```

مهم للذكاء الاصطناعي:

```
degree = dict(G.degree())
```

أو:

```
centrality = nx.degree_centrality(G)
```

عرض الشبكة على الخريطة

تحويل Graph إلى GeoDataFrame

مهم جداً لـ GIS:

الآن لديك:

حفظ البيانات

تحميل الشبكة لاحقاً

تجهيزها للـ GNN

استخراج Features من الطرق

مثال كامل بسيط

```

import osmnx as ox

# تحميل الشبكة
G = ox.graph_from_place("Cairo, Egypt", network_type="drive")

# تحويل إلى GeoDataFrame
nodes, edges = ox.graph_to_gdfs(G)

# عرض
ox.plot_graph(G)

# حفظ
ox.save_graphml(G, "cairo.graphml")

```

ملاحظات مهمة (خبرة عملية)

استخدم `simplify=True` (افتراضي)

لتقليل العقد غير المهمة

استخدم `retain_all=False`

لتجنب الشبكات المنفصلة

استخدم `cache`

```
ox.settings.use_cache = True
```

كيف يفيدك هذا في مشروعك

الشبكة التي حصلت عليها تستخدم في:

- (العلاقات المكانية) GNN
- Network Analysis
- Shortest Path
- Urban Classification

الخطوة التالية (مهمة جداً لك)

بعد تحميل الشبكة مباشرة:

👉 نربطها مع POI
👉 ثم نحولها إلى Graph Dataset

5 تحميل صور الأقمار الصناعية

- Sentinel-2
- أو Google Earth Engine

تحميل صور الأقمار الصناعية (Sentinel-2) بطريقتين عمليتين:

- الطريقة 1: مباشرة ببايثون
- الطريقة 2: باستخدام Google Earth Engine (الأقوى حاليًا)

أولاً: تحميل Sentinel-2 باستخدام Python

سنستخدم مكتبة:

Sentinel Hub 🖱️

تنصيب المكتبات

```
pip install sentinelhub
```

إنشاء حساب والحصول على API

1. ادخل على: <https://apps.sentinel-hub.com>

2. أنشئ حساب

3. احصل على:

client_id ○

client_secret ○

تحديد منطقة الدراسة (Bounding Box)

```
from sentinelhub import BBox, CRS
```

```
bbox = BBox(  
    bbox=[31.10, 29.90, 31.30, 30.10], # (min_lon, min_lat, max_lon, max_lat)  
    crs=CRS.WGS84  
)
```

تحميل الصورة

```
from sentinelhub import SentinelHubRequest, DataCollection, MimeType
```

```
request = SentinelHubRequest(  
    data_folder='data',  
    evalscript="""  
    // RGB bands
```

```

return [B04, B03, B02];
"""
input_data=[
    SentinelHubRequest.input_data(
        data_collection=DataCollection.SENTINEL2_L2A,
        time_interval=('2023-01-01', '2023-01-31'),
    )
],
responses=[
    SentinelHubRequest.output_response('default', MimeType.TIFF)
],
bbox=bbox,
size=(512, 512)
)

data = request.get_data()

```

حفظ الصورة

```

import numpy as np
from PIL import Image

img = np.array(data[0])
Image.fromarray(img).save("sentinel2.png")

```

ثانياً: استخدام Google Earth Engine (الأفضل)
نستخدم:

Google Earth Engine

التسجيل

• <https://earthengine.google.com>
• تفعيل الحساب (قد يستغرق يوم)

تشغيل الكود (Python)

```
pip install earthengine-api geemap
```

تسجيل الدخول

```

import ee
ee.Initialize()

```

تحميل Sentinel-2

```

import ee

# منطقة الدراسة
roi = ee.Geometry.Rectangle([31.10, 29.90, 31.30, 30.10])

# اختيار الصور
collection = (ee.ImageCollection("COPERNICUS/S2_SR")
               .filterBounds(roi)
               .filterDate("2023-01-01", "2023-01-31"))

```

```
.filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 10)))

image = collection.median()
```

عرض الصورة

```
import geemap

Map = geemap.Map()
Map.centerObject(roi, 10)

Map.addLayer(image, {
    'bands': ['B4', 'B3', 'B2'],
    'min': 0,
    'max': 3000
}, 'Sentinel-2')

Map
```

تصدير الصورة

```
task = ee.batch.Export.image.toDrive(
    image=image,
    description='sentinel_export',
    scale=10,
    region=roi.getInfo()['coordinates']
)

task.start()
```

مقارنة بين الطريقتين

الخاصية	Sentinel Hub	Google Earth Engine
سهولة الاستخدام	سهلة	متوسطة
معالجة البيانات	محدودة	قوية جدًا
الفلتر (Cloud, Time)	جيدة	ممتازة
مناسب للأبحاث	جيد	الأفضل

كيف تستخدم الصور في مشروعك

بعد تحميل الصور:

استخراج Patch لكل Node

64×64 أو 128×128

إدخالها إلى CNN

Image → Feature Vector

دمجها مع:

- POI
- Graph

إدخالها إلى GNN أو MLLM

أفضل إعداد لك (نصيحة مهمة)
بما أنك تعمل في:

- GIS
- AI
- Satellite

أنصحك:

استخدم **Google Earth Engine** + تنزيل patches
ثم تدريب locally على (GPU RTX 2070)

الخطوة التالية (مهمة جدًا لك)

الآن عندك:

- شبكة الطرق
- POI
- صور الأقمار الصناعية

الخطوة القادمة:

كيف نربط الصور مع كل Node أو Grid (Patch Extraction) بطريقة احترافية
وهذه خطوة critical في نجاح النموذج

6] اختياري) تحميل بيانات نصية
مثل:

- وصف المناطق
- بيانات حكومية

المرحلة 3: تنظيف وتجهيز البيانات

7] تنظيف POI

- حذف القيم الفارغة
 - توحيد التصنيفات
-

8 توحيد الإحداثيات
كل البيانات في:
WGS84

9 ربط POI بشبكة الطرق
Nearest Node Mapping

10 تقسيم المدينة
إلى:

- (Nodes (Graph-based أو
- (Grid (Hybrid approach

المرحلة 4: بناء Graph

11 إنشاء Graph
(G = (V, E

- V → Nodes
- E → Roads

استخراج العلاقات المكانية

- Neighbor nodes
- Distance
- Road type

المرحلة 6: دمج البيانات (Fusion)

إنشاء Feature Vector موحد
[X = [POI embedding + Image + Graph + Text

هذه هي أهم مرحلة فعليًا في بناء نموذجك:
Feature Fusion (دمج البيانات متعددة الوسائط)

الهدف من المرحلة

نريد تحويل كل المصادر

- POI
- صورة

- شبكة طرق
- نص

Grid أو **Node** واحد موحد لكل **Vector** إلى

$X = [\text{POI embedding} + \text{Image} + \text{Graph} + \text{Text}]$

الفكرة الأساسية

خاص **Encoder** كل نوع بيانات له

النوع	Encoder	الناتج
POI	Word2Vec	Vector
Image	CNN	Vector
Graph	GNN / NetworkX	Vector
Text	BERT	Vector

ثم نقوم بدمجهم في متجه واحد

تجهيز كل **Feature** بشكل منفصل

♦ **A) POI Embedding)**

من **OpenStreetMap**

مثال:

`poi_vector = [0.12, 0.55, 0.88, ...] # 32 dim`

♦ **B) Image Feature)**

باستخدام **CNN** (مثل **ResNet**)

`dim 256 أو image_vector = [0.77, 0.21, 0.63, ...] # 128`

♦ **C) Graph Feature)**

من شبكة الطرق:

`graph_vector = [
 degree,
 centrality,
 clustering
 # مثلاً 3-10 dim`

♦ **D) Text Feature)**

باستخدام **BERT**:

`text_vector = [0.34, 0.91, 0.11, ...] # 384 dim`

توحيد الأبعاد (Very Important)

قبل الدمج يجب:

- Normalize
- Scale
- أو Projection

مثال باستخدام **PyTorch**:

```
import torch.nn as nn

poi_proj = nn.Linear(32, 64)
img_proj = nn.Linear(256, 64)
graph_proj = nn.Linear(10, 32)
(text_proj = nn.Linear(384, 64)
```

طرق الدمج (Fusion Methods)

الطريقة 1: Concatenation (الأبسط)

```
import torch

X = torch.cat([
    poi_vec,
    img_vec,
    graph_vec,
    text_vec
], dim=-1)
النتائج:
Final Feature Vector
```

الطريقة 2: Weighted Fusion (أفضل)

```
X = (
    0.3 * poi_vec +
    0.4 * img_vec +
    0.2 * graph_vec +
    0.1 * text_vec
)
```

الطريقة 3: Attention Fusion (الأقوى)

النموذج يتعلم أهمية كل modality.

```
attention_weights = softmax(W * features)

(X = sum(attention_weights * features
```

الطريقة 4: Gated Fusion (احترافي جدًا)

```
gate = sigmoid(W * concat_features)

X = gate * poi_vec + (1 - gate) * img_vec
```

الشكل النهائي للـ Feature

مثال:

```
Node Feature (Final)

[
    POI: 64 dim
    Image: 64 dim
    Graph: 32 dim
    Text: 64 dim
]

→ Total = 224 dim
```

إدخالها إلى GNN أو MLLM

في GNN:

```
data.x = X
```

في MLLM:

Multi-modal embedding → LLM

مثال كود كامل مبسط

```
import torch

# vectors
poi = torch.randn(32)
img = torch.randn(256)
graph = torch.randn(10)
text = torch.randn(384)

# projection
poi = torch.nn.Linear(32, 64)(poi)
img = torch.nn.Linear(256, 64)(img)
graph = torch.nn.Linear(10, 32)(graph)
text = torch.nn.Linear(384, 64)(text)

# fusion
X = torch.cat([poi, img, graph, text], dim=0)

(print(X.shape))
```

نصائح مهمة (خبرة بحثية)

لا تدمج بدون normalization

```
from sklearn.preprocessing import StandardScaler
```

لا تجعل Dim كبيرة جداً

• الأفضل: 128–512

جرب Ablation Study

- POI فقط
- Image فقط
- POI + Image
- All

أفضل Strategy لك

:

Start:
POI + Image

Then:
+ Graph

Finally:
(MLLM $\mathcal{L}$) + Text

الخلاصة

Fusion Pipeline:

POI $\rightarrow$ Embedding
Image $\rightarrow$ CNN
Graph $\rightarrow$ Features
Text $\rightarrow$ Encoder

$\downarrow$
Projection

$\downarrow$
Fusion (Concat / Attention)

$\downarrow$
Final Feature Vector X

$\downarrow$
GNN / MLLM

المرحلة 7: بناء النماذج الأساسية

(GNN (Spatial Intelligence $\blacklozenge$
باستخدام:

PyTorch Geometric

Vision Encoder $\blacklozenge$

ResNet / EfficientNet $\bullet$

Text Encoder $\blacklozenge$

MiniLM / BERT $\bullet$

المرحلة 8: بناء MLLM صغير (Core Step)

2D اختيار Base Model
مثلاً:

- (LLaMA (small
- أو Distil model

ربط ال Modalities

Graph → GNN → Embedding
Image → CNN → Embedding
Text → Encoder → Embedding

Fusion Layer

Fusion = Concatenation / Attention

إدخالها إلى LLM

MLLM Input = Multi-modal embedding

المرحلة 9: التدريب

إعداد Dataset

(X_multi_modal , Y)

التدريب

- Classification Loss
- أو Contrastive Loss

Fine-tuning

لتحويله إلى:

Geo-Aware Model

المرحلة 10: التقييم

Metrics

- Accuracy
- F1-score
- Spatial Accuracy

المرحلة 11: بناء Digital Twin

- خريطة تفاعلية
- طبقات GIS

إضافة قدرات ذكية

- Query:

"أين المناطق التجارية في القاهرة؟"

المرحلة 12: إخراج Custom Small Geo-MLLM

النموذج النهائي
(Geo-MLLM (Small

يدعم:

- ✓ فهم الصور
- ✓ فهم POI
- ✓ فهم العلاقات المكانية
- ✓ الإجابة على أسئلة جغرافية

الشكل النهائي

OSM (POI + Roads)
+
Satellite Imagery
+
Text Data
↓
Feature Engineering
↓
GNN + CNN + Text Encoder
↓
Fusion Layer
↓
Small LLM
↓
Geo-MLLM
↓
Digital Twin

ملاحظة مهمة جدًا

للحصول على Small Model قوي:

- استخدم:
 - Distillation
 - Quantization

LoRA Fine-tuning ○